

RESEARCH ARTICLE

SCORE: Tokenization-Aware Transformer for Symbolic Music Generation with Expressions

Vishakh Begari*

Abstract

Ever since the electrification of guitar, there has been an explosion in rhythmic complexity, tone experimentation and playing techniques. In this paper, a novel tokenisation method is proposed, which encapsulates the various guitar techniques and a sequence level attention component for the transformer architecture, which tries to capture the segmentation between different musical phrases. The given approach enables efficient batching of variable length musical phrases while preserving their contextual coherence.

Keywords: motif generation, symbolic music, transformers, sequence modelling, KL divergence.

1. Introduction

A *riff* is a short, repeated motif or figure in the melody or accompaniment of a musical composition. Randel (1986) This type of composition is particularly prevalent in rock, metal, blues, funk and jazz. It is often the main feature that gives character and structure to a song. Developments in Transformers architecture have inspired many to generate sequences by treating musical notes as symbolic language Huang et al. (2018) Agostinelli et al. (2023) Copet et al. (2024) Cui et al. (2025) Muhamed et al. (2021) Ferreira et al. (2023) Huang and Yang (2020a) Wu and Yang (2020) Ens and Pasquier (2020) Dong et al. (2023) Yu et al. (2022). Generating modern music demands capturing not only note patterns but also the associated expressions.

A widely adopted method for representing musical notes digitally is the MIDI format Craig (2025), which encodes pitch, duration, and dynamics. Using such a format for musical expression has limitations like resolution and bandwidth constraints Shamoon (2024) when compared to richer representations like audio recordings and detailed scores. Modern guitar riffs frequently employ intense and abrupt expressive techniques, which contribute significantly to its overall identity Vallejo (2021). To accurately represent the nuanced expressive elements of modern music, a score based format is preferable over simpler encodings like MIDI. The proposed method requires a tokenisation scheme that represents notes, beats, beat types and their associated accents, necessitating a dataset that includes expressive performance details.

In the context of symbolic music, notes are represented as sequences of discrete tokens. This requires a

tokenisation scheme which directly affects how effectively a model can learn and generate coherent musical structures. Most existing approaches rely on methods similar to text processing such as BPE (Byte Pair Encoding) Sennrich et al. (2016), WordPiece Schuster and Nakajima (2012) and SentencePiece Kudo and Richardson (2018). Although subword tokenization improves the structure Kumar and Sarmiento (2023) Fradet et al. (2023), it becomes limited when dealing with the expressive element combinations of modern music. Recent studies Gardín (2024) have explored modeling chords based on scale degree, tonality and octave, and notes encoded relative to the current chord. This approach, however, is unable to produce every possible chord, particularly when dissonance is involved, which is becoming increasingly popular in modern metal. Most prior studies use absolute pitch values Suresh et al. (2020), some Kermarec et al. (2022) investigate the use of pitch intervals. This introduces an additional processing step, increasing inference time. Another common strategy treats each unique chord as a separate token, resulting in a large vocabulary that may include thousands of tokens representing all possible chords in the dataset Dalmazzo et al. (2024b) Dalmazzo et al. (2024a). Chen et al. (2020) proposes a pre-defined set of rhythmic groove patterns. This limits the model's ability to generate novel rhythms.

To address these limitations, a tokenisation scheme is proposed, that captures single and multiple musical notes efficiently and their associated accentuated elements. Furthermore, a sequence level attention mechanism is introduced to capture the segmentation between different musical phrases, facilitating batching

*Independent Researcher, Bavaria, Germany

of variable length phrases while maintaining musical context. The modified architecture is trained and evaluated on a dataset Begari (2025a) that includes guitar specific techniques.

While direct comparison with existing methods is not feasible due to differences in model compatibility and expressive feature representation, the proposed method is evaluated using quantitative metrics. The model is trained to minimize KL divergence across pitch, beat, beat type, and expression similarity. The source code of this work is available at Begari (2025c).

2. Proposed Method

2.1 Tokenisation

2.1.1 Note representation

In place of absolute pitch indices, both fret and string numbers are employed to represent notes. This approach allows for a more compact representation of notes, as the same pitch can be played on different strings and frets. This is particularly relevant to guitar music, where notes can be produced at multiple fretboard positions, often resembling chord shapes. The model focuses on learning the relative motion of notes across frets, emphasizing positional changes rather than absolute pitch representation. This provides flexibility to accommodate alternative tunings Weissman (2006). The formula for note encodings for fretted string instrument is given by:

$$n_e = n_i + 2b_t \cdot n_s \cdot n_{ps} + 2b_b \cdot n_s \cdot n_{ps} \quad (1)$$

where, n_s = number of strings, n_{ps} = number of pitch steps per string.

$$n_i = f_n + s_n \cdot n_{ps} + n_s \cdot n_{ps} \cdot n_{val} \quad (2)$$

where, f_n = fret number, s_n = string number and n_{val} = 1 for palm mute, 0 otherwise; b_t and b_b are defined in 1 and 2 respectively.

b_t	Value
Triplet(3)	$1 + d$
Quintuplet(5)	$2 + d$
Sextuplet(6)	$3 + d$
Septuplet(7)	$4 + d$
Nonuplet(9)	$5 + d$
Undecuplet(11)	$6 + d$

Table 1: Beat type. $d = 7$ for dotted note, 0 otherwise.

b_b	Value
$1 / 64$	0
$1 / 32$	1
$1 / 16$	2
$1 / 8$	3
$1 / 4$	4
$1 / 2$	5
1	6

Table 2: Beats.

2.1.2 Accent representation

A description of expressive elements is provided in Begari (2025b). An accent token is appended to the note token from Section 2.1.1 to indicate the presence of an expressive element. For chords, the accent token is added before the next consecutive note token. A C major chord as shown in Figure 1 comprises of 3 notes - root note [C] played on the fret 2 / 5th string, major third [G] on fret 1 / 4th string and perfect fifth [E] on fret 2 / 1st string Alexander (2019). The tokenization for the chord is as follows: 15413, 27051, 15389, 27051, 15321, 27051. Here, 27051 is the chord accent token, BARRE_NOTE¹. Similarly, for a bend note depicted in Figure 2, the tokenization is: 15461, 27052. Here, 27052 is the bend accent token, BEND_NOTE_1¹.

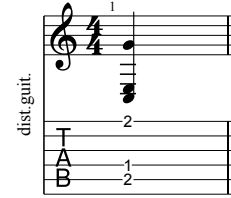


Figure 1: C major chord shape on a 6 string.

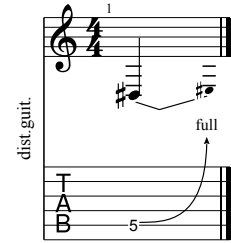


Figure 2: Fret 5 string 5 quarter note full bend.

2.1.3 Phrases

A music bar (also called a measure) is a fundamental unit in musical notation that divides music into segments of equal time, defined by the time signature Miller (2016). Each measure is separated by a BAR¹ token. A collection of measures making a song is separated by an EOS¹ token.

2.2 Embedding

A bar consists of a fixed number of beats, composed of musical notes that were tokenised as described in Section 2.1. Each bar is treated like a sentence. It has been demonstrated that the transformer equipped with relative attention is very well-suited for generative modelling of symbolic music Huang et al. (2018) Zhou et al. (2023) Li et al. (2024). The chunking strategy used to carry over context between sequences during training follows the approach introduced in Yang et al. (2019), which builds upon Transformer-XL’s Dai et al. (2019a) segment-level recurrence mechanism. This method enables the model to capture longer-range dependencies by reusing hidden states from previous token chunks across segments. An example of a riff is shown in is given in Figure 3.

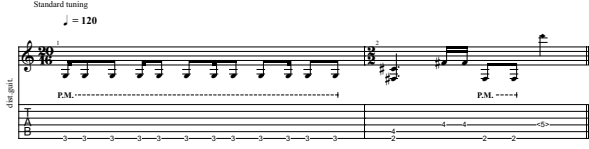


Figure 3: Guitar riff tablature.

With 16 tokens per sequence, embedding is as follows:

[27049, 7985, 11849, 11849, 7985, 11849, 11849, 7985, 11849, 11849, 7985, 11849, 11849, 27049, 17485, 17506]

[11849, 11849, 7985, 11849, 11849, 27049, 17485, 17506, 27051, 7802, 7802, 11848, 11848, 15530, 27074, 27049]

[27051, 7802, 7802, 11848, 11848, 15530, 27074, 27049, 27075, 0, 0, 0, 0, 0, 0, 0]

The first two sequences contain musical notes, while the third sequence is padded with zeros to maintain a consistent length of 16 tokens. This padding ensures that all input sequences are of uniform length, while still allowing the flexibility to handle variable-length musical phrases.

2.3 Training

2.3.1 Dataset

The guitar tracks were obtained from Begari (2025a). The dataset consists of files in .gp5 Abakumov (2025) format in standard tuning. From the full set, a total of 6208 sequences were selected via random sampling to allow efficient training on a standard personal computer. The common practice of 80/20 split was used to create the training and validation sets Goodfellow et al. (2016). For testing, top 10 first notes of each measure in the training set were selected as prompts for sequence generation and evaluation. Relevant statistics about the dataset used for training and evaluation are provided in Begari (2025b).

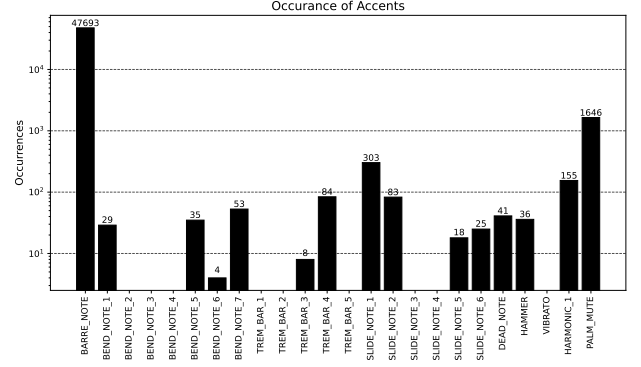


Figure 4: Accent occurrences in training data for a random seed.

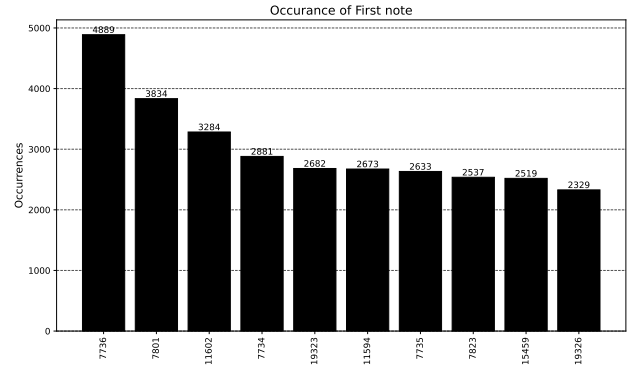


Figure 5: Token occurrences of first note of a musical measure after random seed.

2.3.2 Model

The Transformer is built from a decoder-only stack with masked self attention as described in Huang et al. (2018). Although, segment-level recurrence mechanism is captured in Transformer-XL Dai et al. (2019a), its memory is limited to one previous segment at a time. To model dependencies across a whole batch, the gaussian distributions of the sequences are learned through a Multi layer Perceptron layer. If the input is a batch of sequences defined by:

$$X \in \mathbb{R}^{B \times T \times d} \quad (3)$$

where:

- B : batch size (number of sequences)
 - T : number of tokens per sequence
 - d : embedding dimension
 - $X_i \in \mathbb{R}^{T \times d}$: the i -th sequence in the batch
- Sequence level mean pooling is performed by:

$$\bar{x}_i = \frac{1}{T} \sum_{t=1}^T X_{i,t} \in \mathbb{R}^d \quad (4)$$

Gaussian parameters are then computed with a Multi Layer Perceptron. Let $f: \mathbb{R}^d \rightarrow \mathbb{R}^{2d}$ be a learned MLP. Then:

$$[\mu_i, \log \sigma_i] = f(\bar{x}_i) \quad \text{with} \quad \mu_i, \log \sigma_i \in \mathbb{R}^d \quad (5)$$

Ensure positivity of standard deviation:

$$\sigma_i = \exp(\log \sigma_i) + \varepsilon \quad \text{where } \varepsilon > 0 \quad (6)$$

Each sequence is then represented as a diagonal multivariate Gaussian:

$$\mathcal{N}_i = \mathcal{N}(\mu_i, \text{diag}(\sigma_i^2)) \quad (7)$$

Vectorized for the batch:

$$[\mu, \log \sigma] = f(\tilde{X}) \in \mathbb{R}^{B \times 2d} \quad (8)$$

$$\sigma = \exp(\log \sigma) + \varepsilon \in \mathbb{R}^{B \times d} \quad (9)$$

The pairwise KL divergence between each pair of Gaussians Zhang et al. (2023):

$$\text{KL}(\mathcal{N}_i \parallel \mathcal{N}_j) = \frac{1}{2} \sum_{k=1}^d \left[\log \left(\frac{\sigma_{j,k}^2}{\sigma_{i,k}^2} \right) + \frac{\sigma_{i,k}^2}{\sigma_{j,k}^2} + \frac{(\mu_{i,k} - \mu_{j,k})^2}{\sigma_{j,k}^2} - 1 \right] \quad (10)$$

This produces a KL matrix:

$$\text{KL_matrix} \in \mathbb{R}^{B \times B}, \quad [\text{KL_matrix}]_{i,j} = \text{KL}(\mathcal{N}_i \parallel \mathcal{N}_j) \quad (11)$$

A simpler version of the attention matrix from Dai et al. (2019b) is used from which the KL divergence-based sequence bias is subtracted:

$$\text{RA}_{\text{KL}} = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + QR^\top - \alpha \cdot \text{KL}_{\text{bias}} \right) \quad (12)$$

A weighted sum over relative positional value contributions V_{rel} is then added:

$$\text{Output} = \text{RA}_{\text{KL}} \times V + \text{RA}_{\text{KL}} \times V_{\text{rel}} \quad (13)$$

2.3.3 Loss

The model's loss is composed of three components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{rep}} + \mathcal{L}_{\text{seq}} \quad (14)$$

Cross Entropy loss ($\mathcal{L}_{\text{CE}}$) : Cross-entropy loss is the standard choice for training deep neural networks and logistic regression models for both binary and multi-class classification tasks. ml glossary (2017)

Repetition loss ($\mathcal{L}_{\text{rep}}$) : Repetition helps organise musical ideas, making pieces more coherent and memorable. It gives listeners reference points and expectations, aiding in the recognition and recall of themes or motifs. Hu et al. (2022) Composers have long used stochastic (probability-based) methods to inject randomness into repeated motifs, sometimes even employing quantum randomness for truly unpredictable results. Yamada et al. (2021) This demands careful

consideration, as it relies on the listener's inherent ability to recognise patterns and their natural inclination toward novelty. Since it is already known that accent tokens¹ cannot repeat, it is penalised through repetition loss, making them less likely to be selected again in subsequent steps. A variant of the repetition penalty introduced in Keskar et al. (2019) is employed; however, instead of greedy decoding, multinomial sampling is used to generate tokens to get semantically different output sequences Song et al. (2024). Including this loss during training eliminates the need to explicitly handle constraints for generating valid sequences at inference time.

$$\text{Let } p_{b,t,v} = \text{softmax} \left(\frac{\text{logits}_{b,t,v}}{T} \right) \quad (15)$$

for $b = 1, \dots, B$; $t = 0, \dots, T-1$; $v = 1, \dots, V$

Compute average past probabilities:

$$\bar{p}_{b,t,v} = \frac{1}{t} \sum_{k=0}^{t-1} p_{b,k,v}, \quad \text{for } t \geq 1 \quad (16)$$

Repetition score (dot product of current and average previous probabilities)

$$R_{b,t} = \sum_{v=1}^V p_{b,t,v} \cdot \bar{p}_{b,t,v}, \quad \text{for } t \geq 1 \quad (17)$$

Final penalty (mean repetition score across batch and sequence, excluding)

$$\mathcal{L}_{\text{rep}} = \frac{\lambda}{B(T-1)} \sum_{b=1}^B \sum_{t=1}^{T-1} R_{b,t} \quad (18)$$

where λ is a repetition penalty weight hyper parameter.

Sequence Penalty loss ($\mathcal{L}_{\text{seq}}$) : Pairwise KL divergence between each pair of Gaussians is given by (10). Given a batch of B sequences, Sequence loss is defined by:

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_{b=1}^B \text{KL}(\mathcal{N}_q^{(b)} \parallel \mathcal{N}_k^{(b)}) \quad (19)$$

To make the training more stable, the sequence loss is annealed and clamped.

$$\mathcal{L}_{\text{seq}} = \min \left(\frac{\text{step}}{\text{EPOCHS}} \cdot \left(\frac{\delta \cdot \mathcal{L}_{\text{KL}}}{\text{BATCH}} \right), 0.2 \right) \quad (20)$$

where, δ is a sequence weight hyper parameter.

3. Results

The idea of context carryover has been well established in Dauphin et al. (2016) Wei et al. (2021) Chen et al. (2025) Rae et al. (2019) Beltagy et al. (2020) Zaheer et al. (2021). During inference, the model carries over

context from previous sequences by using the last token of the preceding measure as the first token for the subsequent measure, following the approach described in Huang and Yang (2020b). To compare the quality of predicted tokens (P) from true distributions (Q), relative entropy is calculated using Kullback-Leibler divergence, following the methodology in Seo et al. (2016) Zhang et al. (2025) Spiliopoulou and Storkey (2011):

$$D_{KL}(P \parallel Q) = \sum_{x \in X} P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (21)$$

3.1 Grid search

To determine the optimal configuration, a grid search was performed over a predefined set of hyperparameter values. The search included combinations of key parameters such as learning rate, sequence length, model dimensionality, Each configuration was trained for 10 epochs, and performance was evaluated on a validation set using loss. The best-performing set of hyperparameters was selected based on the lowest validation score.

The reported loss values in Table 4 were computed as the mean over five independent training runs after 100 epochs for hyper-parameter values listed in Table 3.

Property	Value
FFN_HIDDEN	512
MAX_SEQ_LENGTH	16
NUM_HEADS	4
DROP_PROB	0.3
NUM_LAYERS	3
D_MODEL	512
LEARNING_RATE	0.0001
TEMPERATURE	1.0
OVERLAP	8
LAMBDA	0.5
ALPHA	0.1
DELTA	0.001

Table 3: Hyper-parameter values used for training.

Given that the model is autoregressive and expects an initial token, selecting the start token based on the distribution of starting notes of musical measures in the training data yields more consistent generation as shown in Table 5. The results were averaged over 10 generated phrases per prompt.

Loss	Mean	Std dev
Cross Entropy	1.458764	0.044821
Repetition	0.033093	0.000869
Sequence	0.199693	0.000002
Total	1.691551	0.045377
Validation	1.824288	0.116141

Table 4: Loss values.

Token	Count	Notes	Beat	Beat-type
15528	4748	0.3375	0.4568	0.3127
7734	3556	0.2137	0.55	0.3908
7732	3497	1.4628	0.5108	0.7077
23304	2989	2.2162	1.4351	0.6563
7756	2064	2.0098	1.8383	0.8837

Table 5: KL Divergence for notes, beats and beat type.

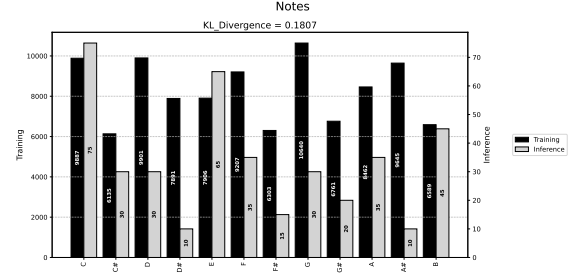


Figure 6: KL Divergence for musical notes with start token 15528.

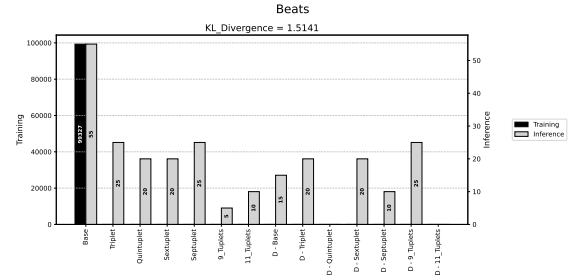


Figure 7: KL Divergence for beat with start token 15528.

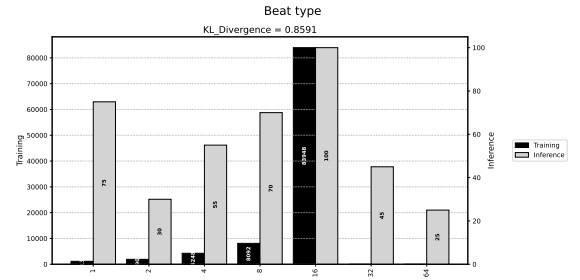


Figure 8: KL Divergence for beat type with start token 15528.

4. Conclusions

The proposed tokenisation scheme efficiently incorporates expressive elements without needing an explicit representation like chord vocabulary. The sequence-level attention mechanism captures phrase segmentation, enabling variable-length batching while preserving musical coherence. The model demonstrates strong performance in generating guitar riffs with expressive details, as evidenced by low KL divergence scores

across pitch, beat, and beat type distributions. By incorporating a repetition penalty loss, the model naturally reduces redundancy without requiring explicit inference constraints. Future work could explore extending the approach to other instruments and incorporating additional musical attributes.

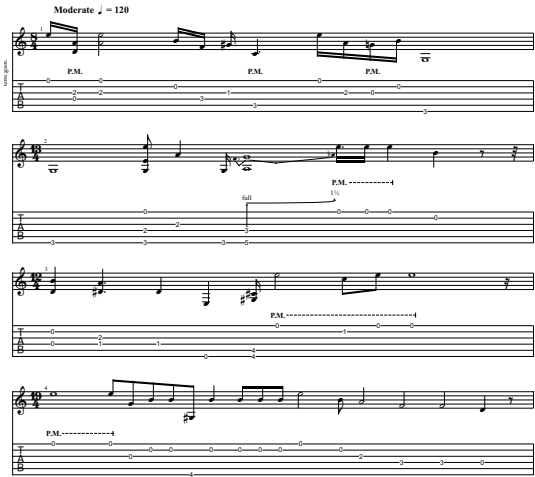


Figure 9: Test results showing 4 musical measures generated with TEMPERATURE = 0.7

Notes

1

Accent	Token ID
BAR	27049
BOS	27050
BARRE_NOTE	27051
BEND_NOTE_1	27052
BEND_NOTE_2	27053
BEND_NOTE_3	27054
BEND_NOTE_4	27055
BEND_NOTE_5	27056
BEND_NOTE_6	27057
BEND_NOTE_7	27058
TREM_BAR_1	27059
TREM_BAR_2	27060
TREM_BAR_3	27061
TREM_BAR_4	27062
TREM_BAR_5	27063
DEAD_NOTE	27064
SLIDE_NOTE_1	27065
SLIDE_NOTE_2	27066
SLIDE_NOTE_3	27068
SLIDE_NOTE_4	27069
SLIDE_NOTE_5	27070
SLIDE_NOTE_6	27071
HAMMER	27072
VIBRATO	27073
HARMONIC_1	27074
EOS	27075

Table 6: Token IDs for various accents.

Acknowledgements

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

References

Abakumov, S. (2025). gpro file format (.gp3, .gp4, .gp5, .gpx). pyguitarpro Documentation. Accessed: 2025-11-07.

Agostinelli, A., Denk, T. I., Borsos, Z., Engel, J., Verzetti, M., Caillon, A., Huang, Q., Jansen, A., Roberts, A., Tagliasacchi, M., Sharifi, M., Zeghidour, N., and Frank, C. (2023). Musiclm: Generating music from text. <https://arxiv.org/abs/2301.11325>.

Alexander, J. (2019). *Guitar Chords in Context: The Practical Guide to Chord Theory and Application*. Fundamental Changes.

Begari, V. (2025a). Score-set. <https://github.com/DjntleViBe/SCORE-SET>. Accessed: 2025-07-11.

Begari, V. (2025b). Score-set: A dataset of guitarpro files for music phrase generation and sequence learning. <https://arxiv.org/abs/2507.18723>.

Begari, V. (2025c). Supervised composition and riff engine. <https://github.com/DjntleViBe/SCORE>. Accessed: 2025-07-11.

Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. <https://arxiv.org/abs/2004.05150>.

Chen, Y., You, Z., Zhang, S., Li, H., Li, Y., Wang, Y., and Tan, M. (2025). Core context aware transformers for long context language modeling. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J., editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 9691–9710. PMLR.

Chen, Y.-H., Huang, Y.-H., Hsiao, W.-Y., and Yang, Y.-H. (2020). Automatic composition of guitar tabs by transformers and groove modeling. *arXiv preprint arXiv:2008.01431*.

Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., and Défossez, A. (2024). Simple and controllable music generation. <https://arxiv.org/abs/2306.05284>.

Craig, P. (2025). Standard midi file structure. <https://ccrma.stanford.edu/~craig/14q/midifile/MidiFileFormat.html>. Accessed: 2025-11-04.

Cui, X. H., Hu, P., and Huang, Z. (2025). Music sequence generation and arrangement based on transformer model. *Journal of Computational Methods in Sciences and Engineering*, 25(5):3986–3999.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J. G., Le, Q. V., and Salakhutdinov, R. (2019a). Transformer-xl:

- Attentive language models beyond a fixed-length context. *CoRR*, abs/1901.02860. <http://arxiv.org/abs/1901.02860>.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J. G., Le, Q. V., and Salakhutdinov, R. (2019b). Transformer-xl: Attentive language models beyond a fixed-length context. *CoRR*, abs/1901.02860. <http://arxiv.org/abs/1901.02860>.
- Dalmazzo, D., Deguernel, K., and Sturm, B. (2024a). The chordinator: Modeling music harmony by implementing transformer networks and token strategies.
- Dalmazzo, D., Déguernel, K., and Sturm, B. L. T. (2024b). The chordinator: Modeling music harmony by implementing transformer networks and token strategies.
- Dauphin, Y. N., Fan, A., Auli, M., and Grangier, D. (2016). Language modeling with gated convolutional networks. *CoRR*, abs/1612.08083. <http://arxiv.org/abs/1612.08083>.
- Dong, H.-W., Chen, K., Dubnov, S., McAuley, J., and Berg-Kirkpatrick, T. (2023). Multitrack music transformer. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE.
- Ens, J. and Pasquier, P. (2020). Mmm: Exploring conditional multi-track music generation with the transformer. *arXiv preprint arXiv:2008.06048*.
- Ferreira, P., Limongi, R., and Fávero, L. P. (2023). Generating music with data: Application of deep learning models for symbolic music composition. *Applied Sciences*, 13(7).
- Fradet, N., Gutowski, N., Chhel, F., and Briot, J.-P. (2023). Byte pair encoding for symbolic music.
- Gardín, A. (2024). The musiclang tokenizer: Toward controllable symbolic music generation. <https://musiclang.github.io/tokenizer/>. Accessed: 2025-11-04.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 7, Section 5: Training, Validation, and Test Sets.
- Hu, Z., Ma, X., Liu, Y., Chen, G., and Liu, Y. (2022). The beauty of repetition in machine composition scenarios. In *Proceedings of the 30th ACM International Conference on Multimedia*, MM '22, pages 1223–1231. ACM.
- Huang, C. A., Vaswani, A., Uszkoreit, J., Shazeer, N., Hawthorne, C., Dai, A. M., Hoffman, M. D., and Eck, D. (2018). An improved relative self-attention mechanism for transformer with application to music generation. *CoRR*, abs/1809.04281. <http://arxiv.org/abs/1809.04281>.
- Huang, Y.-S. and Yang, Y.-H. (2020a). Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions. In *Proceedings of the 28th ACM International Conference on Multimedia*, MM '20, page 1180–1188, New York, NY, USA. Association for Computing Machinery.
- Huang, Y.-S. and Yang, Y.-H. (2020b). Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions. MM '20, page 1180–1188, New York, NY, USA. Association for Computing Machinery.
- Kermarec, M., Bigo, L., and Keller, M. (2022). Improving tokenization expressiveness with pitch intervals.
- Keskar, N. S., McCann, B., Varshney, L. R., Xiong, C., and Socher, R. (2019). CTRL: A conditional transformer language model for controllable generation. *CoRR*, abs/1909.05858. <http://arxiv.org/abs/1909.05858>.
- Kudo, T. and Richardson, J. (2018). Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. <https://arxiv.org/abs/1808.06226>.
- Kumar, A. and Sarmiento, P. (2023). From words to music: A study of subword tokenization techniques in symbolic music generation. <https://arxiv.org/abs/2304.08953>.
- Li, P., Wu, J., and Ji, Z. (2024). Hrpe: Hierarchical relative positional encoding for transformer-based structured symbolic music generation. In *Music Intelligence*, volume Cr16 of *Communications in Computer and Information Science*, pages 122–134. Springer Nature Singapore.
- Miller, M. (2016). *Music Theory*, 3E. O'Reilly Media. Electronic version available via O'Reilly platform.
- ml glossary (2017). Machine learning glossary. https://ml-cheatsheet.readthedocs.io/en/latest/loss_functions.html.
- Muhammed, A., Li, L., Shi, X., Yaddanapudi, S., Chi, W., Jackson, D., Suresh, R., Lipton, Z. C., and Smola, A. J. (2021). Symbolic music generation with transformer-gans. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35:408–417.
- Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillcrap, T. P. (2019). Compressive transformers for long-range sequence modelling. <https://arxiv.org/abs/1911.05507>.
- Randel, D. M. (1986). *New Harvard Dictionary of Music*. Harvard University Press., Cambridge, MA.
- Schuster, M. and Nakajima, K. (2012). Japanese and korean voice search.
- Sennrich, R., Haddow, B., and Birch, A. (2016). Neural machine translation of rare words with subword units. <https://arxiv.org/abs/1508.07909>.
- Seo, J. S., Kim, J., and Park, J. (2016). Centroid-model based music similarity with alpha divergence. *The Journal of the Acoustical Society of Korea*, 35(2):83–91.
- Shamoon, E. (2024). What's wrong with midi? <https://www.perfectcircuit.com/signal/whats-wrong-with-midi>.

- 463 Song, Y., Wang, G., Li, S., and Lin, B. Y. (2024).
 464 The good, the bad, and the greedy: Evaluation of
 465 llms should not ignore non-determinism. <https://arxiv.org/abs/2407.10457>.
 466
- 467 Spiliopoulou, A. and Storkey, A. (2011). Comparing
 468 probabilistic models for melodic sequences. <https://arxiv.org/abs/1109.6804>.
 469
- 470 Suresh, R., Muhamed, A., Li, L., Cavins, P., Reza-
 471 poor, M., Chi, W., and Yaddanapudi, S.
 472 (2020). Using transformers to create music
 473 in aws deepcomposer music studio. [https://aws.amazon.com/blogs/machine-learning/](https://aws.amazon.com/blogs/machine-learning/using-transformers-to-create-music-in-aws-deepcomposer-music-studio/)
 474 [using-transformers-to-create-music-in-aws-deepcomposer-music-studio/](https://aws.amazon.com/blogs/machine-learning/using-transformers-to-create-music-in-aws-deepcomposer-music-studio/).
 475
- 476 Vallejo, A. (2021). Development, mechanics and com-
 477 positional uses of virtuosic electric guitar tech-
 478 niques. Master’s thesis, University of Huddersfield.
 479 Accessed: 2025-11-04.
- 480 Wei, K., Tran, T., Chang, F.-J., Sathyendra, K. M., Mu-
 481 niyappa, T., Liu, J., Raju, A., McGowan, R., Su-
 482 sanj, N., Rastrow, A., and Strimel, G. P. (2021).
 483 Attentive contextual carryover for multi-turn end-
 484 to-end spoken language understanding. <https://arxiv.org/abs/2112.06743>.
 485
- 486 Weissman, D. (2006). *Guitar Tunings: A Comprehensive*
 487 *Guide*. Routledge.
- 488 Wu, S.-L. and Yang, Y.-H. (2020). The jazz transformer
 489 on the front line: Exploring the shortcomings of
 490 ai-composed music through quantitative measures.
 491 *arXiv preprint arXiv:2008.01307*.
- 492 Yamada, R., Grandi, S., Muñoz-Gil, G., Barbiero, L.,
 493 Aloy, A., and Lewenstein, M. (2021). Applications
 494 of quantum randomness: From rabi oscillations to
 495 fourier axis controlling the musical timbre. *Inter-*
 496 *national Journal of Music Science, Technology and*
 497 *Art*, 3(2):17–25.
- 498 Yang, Z., Dai, Z., Yang, Y., Carbonell, J. G., Salakhut-
 499 dinov, R., and Le, Q. V. (2019). Xlnet: Gen-
 500 eralized autoregressive pretraining for language
 501 understanding. *CoRR*, abs/1906.08237. <http://arxiv.org/abs/1906.08237>.
 502
- 503 Yu, B., Lu, P., Wang, R., Hu, W., Tan, X., Ye, W.,
 504 Zhang, S., Qin, T., and Liu, T.-Y. (2022). Muse-
 505 former: Transformer with fine-and coarse-grained
 506 attention for music generation. *Advances in neural*
 507 *information processing systems*, 35:1376–1388.
- 508 Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J.,
 509 Alberti, C., Ontanon, S., Pham, P., Ravula, A.,
 510 Wang, Q., Yang, L., and Ahmed, A. (2021). Big
 511 bird: Transformers for longer sequences. <https://arxiv.org/abs/2007.14062>.
 512
- 513 Zhang, H., Liang, J., Phan, H., Wang, W., and Bene-
 514 tos, E. (2025). From aesthetics to human pref-
 515 erences: Comparative perspectives of evaluating
 516 text-to-music systems. <https://arxiv.org/abs/2504.21815>.
 517
- 518 Zhang, Y., Pan, J., Li, L. K., Liu, W., Chen, Z., Liu,
 519 X., and Wang, J. (2023). On the properties of
 kullback-leibler divergence between multivariate
 gaussian distributions. In Oh, A., Naumann, T.,
 Globerson, A., Saenko, K., Hardt, M., and Levine,
 S., editors, *Advances in Neural Information Process-*
ing Systems, volume 36, pages 58152–58165. Cur-
 ran Associates, Inc.
 Zhou, J., Zhu, H., and Wang, X. (2023). Choir
 transformer: Generating polyphonic music with
 relative attention on transformer. *arXiv preprint*
arXiv:2308.02531.